

PackageTriage

A decision-driven migration engine
for SSIS packages

— *in active development* —

A .dtsx package in — a control-flow graph, a quantified
risk report and a human-review workbook out.
Every trap found, flagged and priced.

*Validated on Microsoft's real WideWorldImporters
ETL package — 123 objects, 43 review findings in ~3 seconds*

Malin Jacobsson

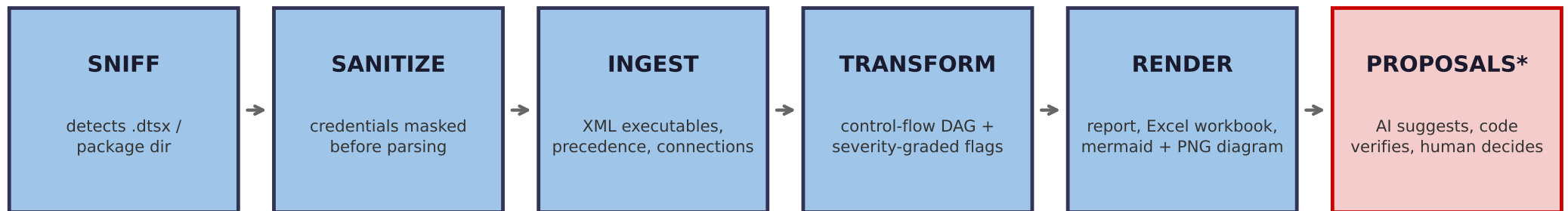
The problem

SSIS packages are XML soup: tasks, containers, precedence constraints, expressions and connections — all invisible until you open the designer. Script tasks hide .NET code, expressions decide flow at runtime, and project-level connection strings live in .conmgr files outside the package.

Before anyone can price a re-platform (ADF, Fabric, dbt, Python), somebody spends days just answering:
what does this package actually DO?

PackageTriage answers that in seconds — from the .dtsx file alone. No SSIS runtime required.

How it works



The first five stages are deterministic — same input, same output.
GUID-based connection references are resolved against adjacent
.conmgr files; what the package cannot see stays flagged.
Proposals are opt-in behind a backend interface.

Outputs: *_canonical.json · *_out.json · *_dag.md · *_dag.png ·
*_review.md · *_hitl.xlsx · *_decisions.json · *_run_report.json

Result on a real package

WideWorldImporters DailyETLMain.dtsx (Microsoft's sample ETL project) — an 880 KB production-style package: sequence containers, data flows, SQL tasks, expressions.

123 objects · 236 edges

tasks, containers, data flows, connections — mapped

14 sequence containers + nested executables

the real structure, not a flat task list

15 expression-driven tasks

runtime values the re-platform cannot guess

2 project-level connections

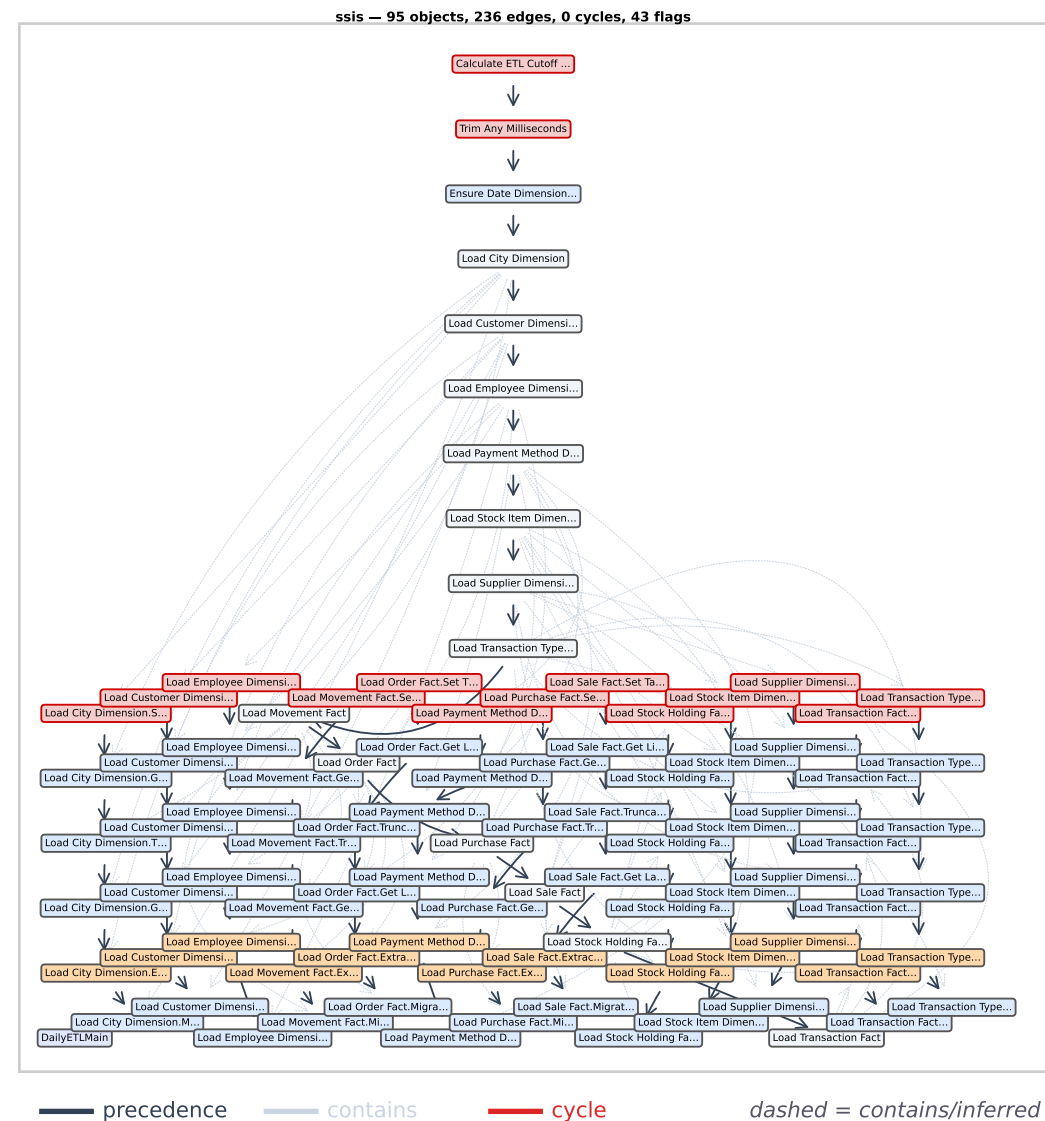
strings live in .conmgr — flagged, resolved via DTSID

43 review findings

the workbook IS the work order

Full pipeline run: ~3 seconds for an 880 KB package.

Synthetic regression fixtures cover all 13 flag rules.



The review loop

Reviewing IS migrating — each decision mutates the graph and regenerates the artifacts, not just the report.

Every warn flag and dropped item becomes a numbered decision

decisions.json — stable ids,
survives re-runs

Click a flag — see where the problem lives

canonical source slice +
the DAG node it lands on

Buttons say the outcome, not the process

Break cycle · Dead task — drop ·
Map expressions · Confirmed

Zero-dependency, offline, runs where the data lives

the Excel workbook stays as
the exportable record

Drop a precedence edge or a dead task — the graph rebuilds in ~1 second and the cycle visibly disappears.

The control-flow graph — before / after a decision

Layered precedence view

topological order — what runs first is on top

Solid = precedence, dotted = containment

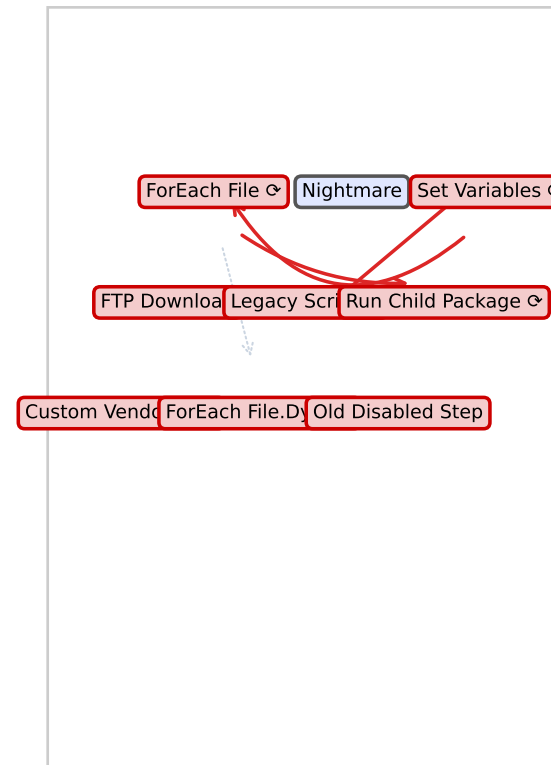
connection and data references live in canonical, not the map

Flagged objects tinted red

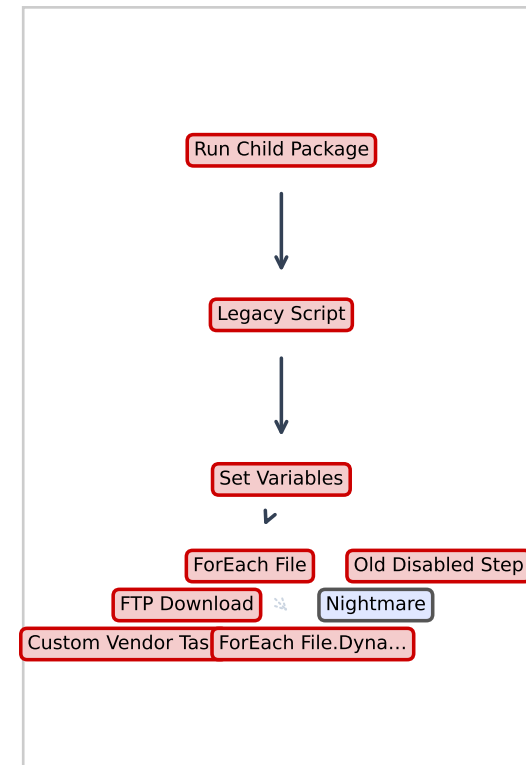
cycles marked ⚡ — the map doubles as a risk radar

Rebuilt after every decision

ignore an edge and the cycle visibly breaks — watch it untangle



BEFORE — 1 cycle



AFTER — cycle broken

— precedence — contains — cycle

decision: ignore the first precedence edge — cycles recomputed to 0, the flag resolves itself

What's left for a human

The honest backlog — what static analysis provably cannot resolve, counted live.

Script tasks & components

embedded .NET code cannot be auto-migrated —
extract the logic, rewrite in the target

Runtime expressions

expression-driven tasks and conditional precedence
resolve values at runtime — map each by hand

Event handlers & loops

control flow that does not follow the main graph —
iteration and error handling need explicit mapping

External integrations

FTP, child packages, OS commands, project-level
connections — each needs an orchestrator equivalent

*Empty queue + empty manual work = a finished,
reproducible migration spec.*

Design principles

Nothing is hidden

Tasks the parser sees but cannot fully resolve are kept and flagged — the chaos stays visible.

Provenance is structural

Observed facts, inferred edges, dropped constructs and AI proposals are separate channels — what the engine KNOWS vs GUESSES.

AI proposes — never decides

An optional backend may suggest resolutions. Code verifiers check every proposal against the model; the human approves.

Runs where the data lives

Deterministic, offline-capable, no telemetry — reads files, never runs the package.